

The paper-latch challenge

Lesson 018 — keypad, color, light, and a tiny state machine

Each new connection unlocks the next visible trick.

Time	about 60 minutes	Board	Arduino Mega 2560
Payoffs	key light, color states, paper latch, optional words	Energy	E1, USB only
Part	authorized 4x4 membrane keypad	Motion	none

This is a tabletop story machine, not a lock. A good four-key pattern makes a green light and turns on one LED behind a paper drawing of a latch. A wrong pattern makes amber. Three wrong patterns begin a short lockout. Nothing moves, protects property, checks a real identity, or controls a door.

Stop rule. Unplug USB before moving wires. Give every LED channel its own resistor of at least $220\ \Omega$. Stop for heat, smell, smoke, repeated USB resets, or any part whose pinout does not match its own documentation.

Prerequisites. Lessons 001–017, especially Lesson 016’s keypad identification and event vocabulary. Lesson 017’s separation between program intent and physical effect carries forward; its servo and pointer do not.

Gather the parts

Mega 2560, USB cable, authorized eight-tail 4x4 membrane keypad, breadboard, common-cathode RGB LED, ordinary LED, four $220\ \Omega$ or larger resistors, jumpers, tape, pencil, and a small paper latch drawing.

The LCD is a bonus stage. Use it only if it is a separately identified, bare parallel HD44780-compatible LCD1602 with the literal 16-pin order shown later. An I2C backpack, an unknown pin order, or an unknown backlight stays out of this build. The whole project remains fun and complete without the LCD.

Adventure map

keypad + D13 → RGB story → paper-latch LED → optional LCD words → denial, lockout, recovery

Use the canonical `Lesson018AccessTrainer` sketch throughout. It is configured with the harmless teaching pattern 1–2–3–4. Star clears an unfinished entry; Hash submits it. The adventure has a fixed two-minute deadline: it then blanks its visible outputs, holds that inactive command for 250 ms, releases resources in reverse order, and waits for reset. Key presses cannot postpone or skip that ending.

Stage 1: make the keypad answer

The authorized part has sixteen keys and eight tail conductors. This project uses its familiar twelve-key area: rows R0–R3 and columns C0–C2. Identify all eight conductors with USB removed, using the continuity method from Lesson 016. Tape C3—the A/B/C/D column conductor—by itself. It touches no Mega pin, 5 V, GND, rail, or neighboring conductor.

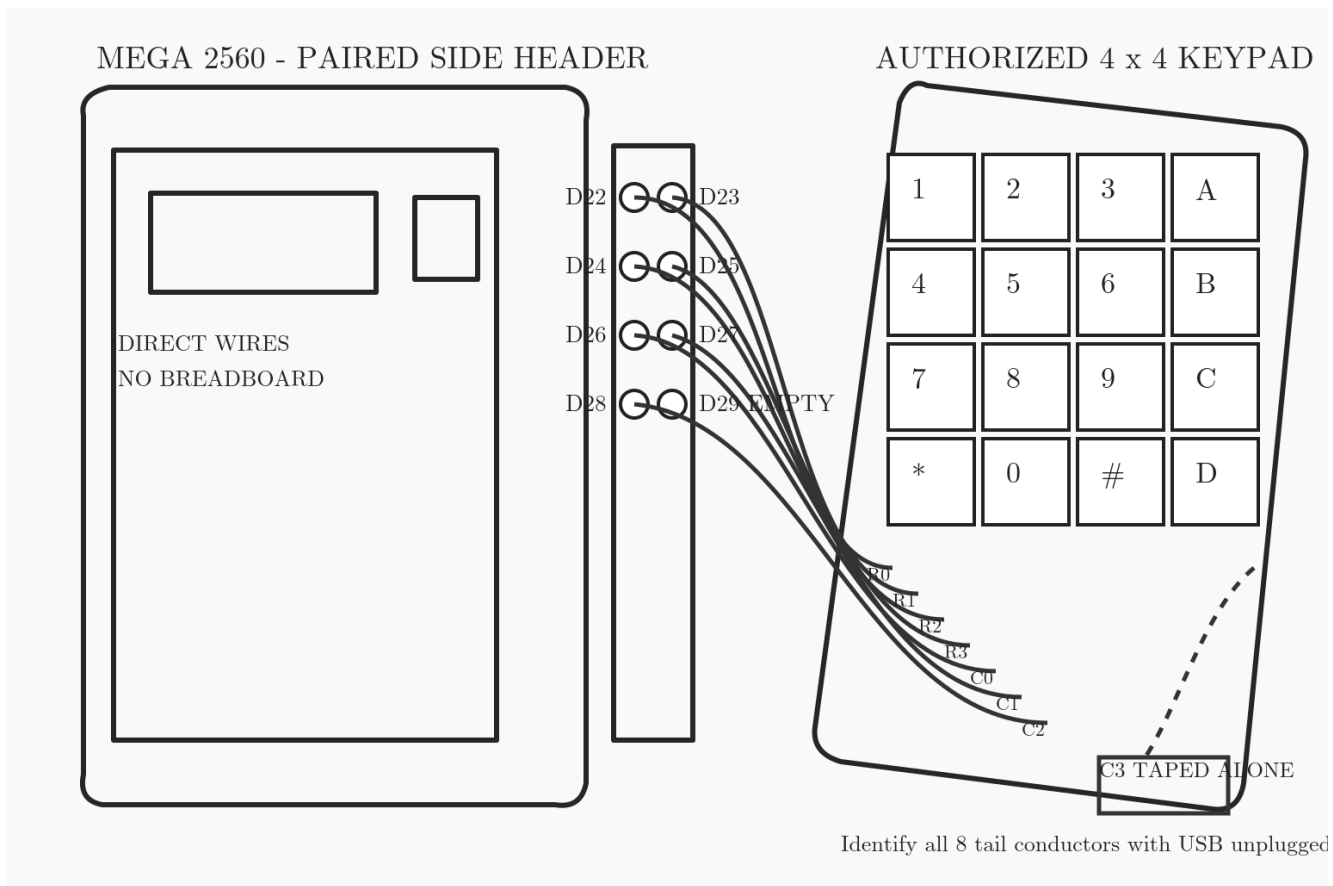


Plate 1: actual paired header and keypad. D22–D25 are R0–R3; D26–D28 are C0–C2. D29 is empty. C3 is individually insulated.

Upload the sketch. D13 flashes for about a quarter second only after the project has acquired its pins. Press 1, then Star, then 1 again. The later indicators will turn that same clean key event into a visible story. If D13 never flashes, unplug USB and compare every one of the seven active conductors with Plate 1; adding more parts cannot repair a wrong keypad map.

Stage 2: let color narrate the entry

Place a common-cathode RGB LED across the breadboard center gap. Confirm its leg order from the part documentation: RGB packages are not all alike. Connect D5, D6, and D7 to the three color legs through three separate resistors. Connect the common cathode to the negative rail, then join that rail to Mega GND. Never copy a guessed leg order from the drawing.

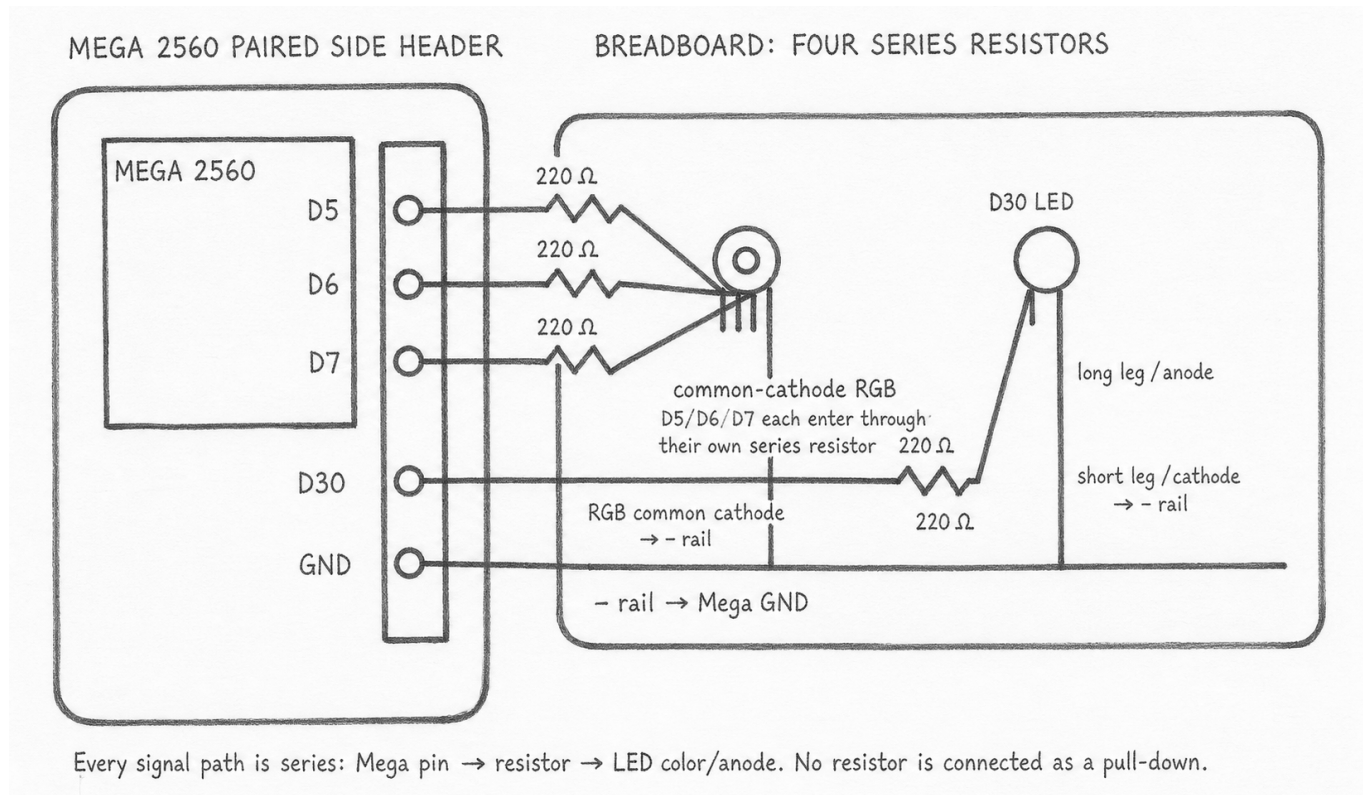


Plate 2: actual paired header and breadboard paths. The left LED is the RGB package; its three resistor paths return through one confirmed common cathode. The right LED belongs to Stage 3.

Restore USB. Blue means ready. Each accepted digit changes the state to white. Star returns to blue. The color change is the reward and the diagnosis: if the keypad is mapped correctly but one channel is absent, check that channel's resistor path and the package's documented common leg.

Stage 3: build a latch that cannot pinch

Draw a tiny latch on paper and tape an ordinary LED behind its “open” mark. Connect D30 through its own resistor to the LED's long leg; connect the short leg to the negative rail as Plate 2 shows. This is a lighted paper prop: D30 means only “the program is showing open intent.”

Type 1-2-3-4 and press Hash. Green and the D30 LED appear together for three seconds, then both return automatically to ready. A missing D30 reward now points to only one small branch: D30, its resistor, LED orientation, or the ground rail. Do not add a servo, relay, motor, solenoid, lock, or external power supply.

Stage 4: optional words on the screen

Only build this stage for the exact parallel LCD identity described in the parts section. The base sketch defaults `ADK_LESSON018_USE_LCD` to 0, so an absent screen cannot block the keypad, RGB, or paper latch. For this exact display, change that one definition to 1, compile again, and then wire Plate 3. RGB and D30 still tell the complete story if the LCD stage is skipped.

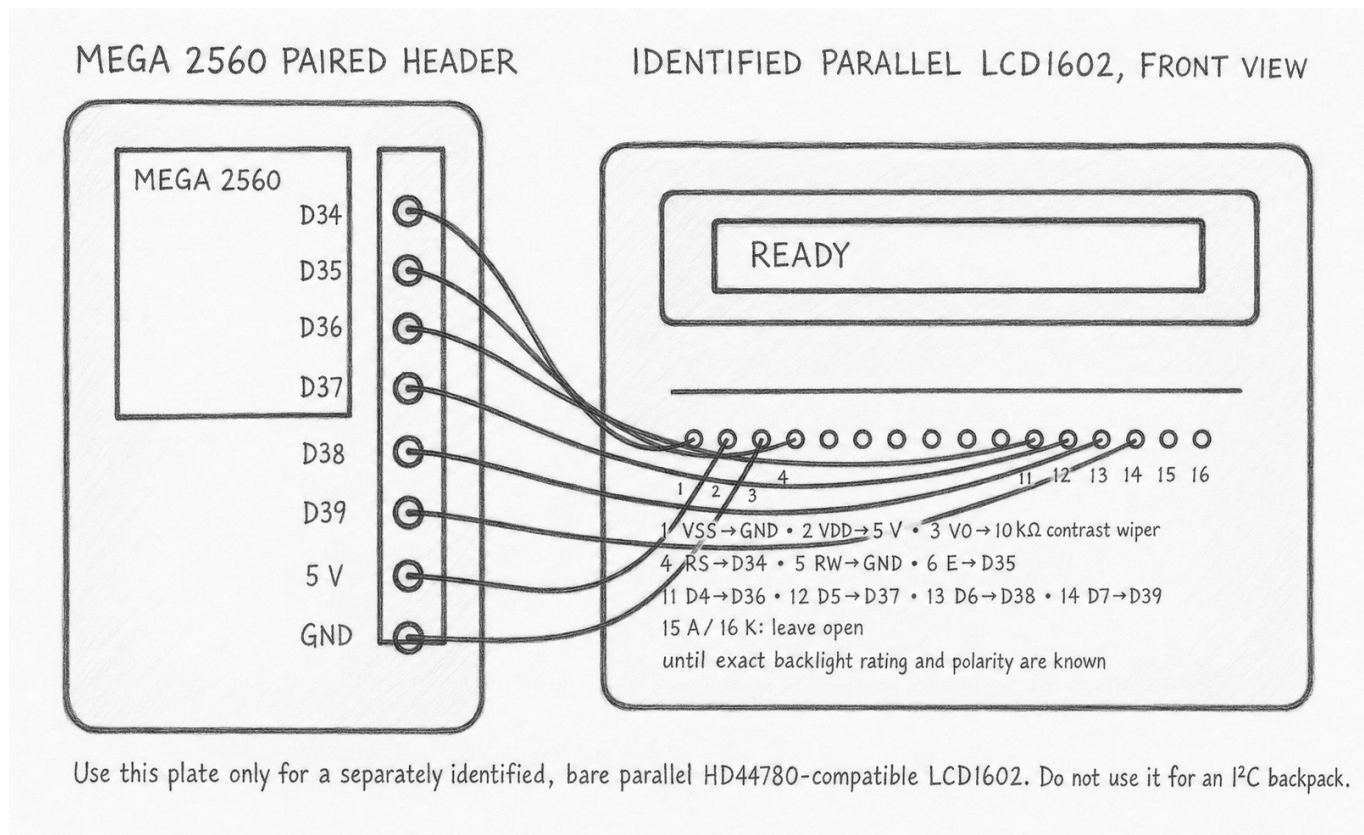


Plate 3: literal LCD1602 header. LCD RS/E/D4/D5/D6/D7 go to Mega D34/D35/D36/D37/D38/D39. VSS and RW go to GND; VDD goes to 5 V. VO is the wiper of a correctly wired 10 kΩ contrast potentiometer. Leave A/K open until the exact module's backlight polarity and current limit are known.

Power up. After LCD startup, row one should say **READY**; the second row shows the entry count. If the indicators work but the screen is blank, turn power off and check identity, contrast wiring, RS/E order, and D4–D7 order. Do not disturb the already-working keypad and indicators.

Stage 5: discover the whole little drama

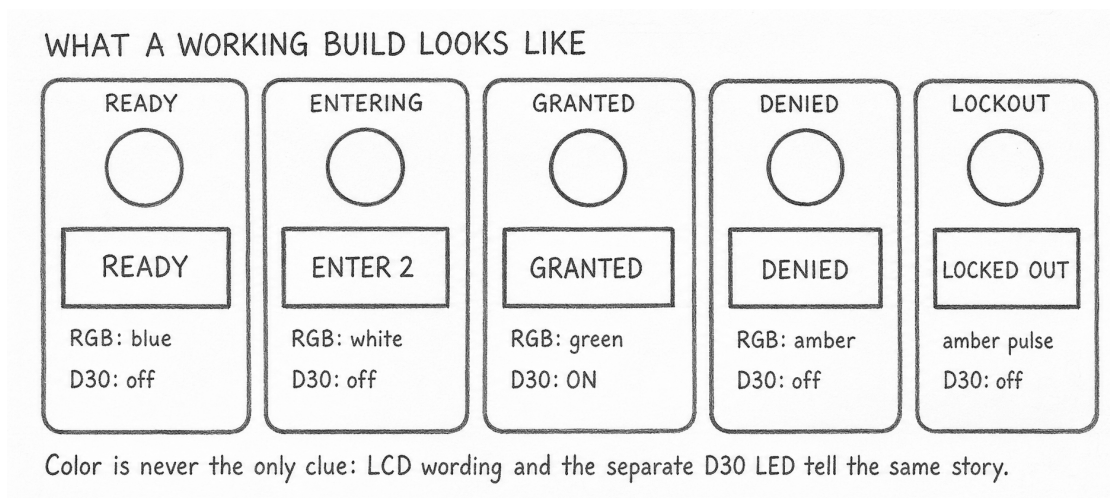


Plate 4: expected visible states. The words and D30 state repeat the meaning of color, so color is never

the only clue.

1. Enter 1–2, press Star, and watch white return to blue.
2. Enter 1–2–3–4, press Hash, and enjoy green plus the paper latch.
3. Enter 1–1–1–1 and submit. Amber means denied; it clears after one second.
4. Do that three times. Amber now pulses for ten seconds while D30 stays off. Key presses cannot skip the pause.
5. Wait. Blue **READY** returns by itself, and the good pattern works again.
6. Before two minutes expire, try any keys you like. At the fixed deadline, every visible output goes inactive; after a quarter second the pins release. Press Reset for another adventure.

This sequence is hard to fake accidentally: the green/D30 payoff requires four correct debounced key events followed by Hash. A wrong map, held key, two-key chord, missing ground, or failed output prevents the later reward rather than awarding it anyway.

When the story stops

What you see	Smallest useful move
No short D13 flash	unplug; compare the seven keypad wires with Plate 1
D13 flashes, no color	check RGB identity, three resistors, and common ground
Color works, D30 does not	check only the D30 resistor/LED branch
Indicators work, LCD blank	keep the successful circuit; check exact LCD identity, contrast, and Plate 3
Red FAULT	release all keys, unplug, inspect for a chord or loose wire, then reconnect; the project restarts closed
Amber keeps pulsing	wait ten seconds; this is the intended lockout story

Unplugging USB ends the adventure. The program turns indicators off before releasing its pins whenever its normal shutdown path runs. After a fault, remove power, correct the one bad connection, and restart from Stage 1's D13 flash. There is no instructor sign-off and no learner test sheet.

Change the story

Try one change at a time: choose a different four-key teaching pattern that is not a real credential; shorten the green reward; rename the LCD prompts; or draw a new paper prop. Keep the prop inert and keep D30 described as a program intent, not proof that something physical happened.

Companion material. The HTML page carries searchable pin tables and source links. This printable lesson carries the staged wiring plates and visible story. Automated host checks remain behind the scenes in the repository; the learner progresses by making each circuit reward work.